FixFirst Project Proposal

A Python Troubleshooting and Repair Verification Tool

NUS-ISS Intelligent Reasoning Systems Practice Module

11 September 2026

Practice Module Group 24

Group member	**Student ID**
HE ZIHANG	A0xxxx72R
WEI YI	A0xxxx64H
CHI YONGJUN	A0xxxx66A



Figure 1 From scattered errors to guided actions and checked results

Introduction

FixFirst will help developers handle errors in small Python projects. When package versions, tests and code checks fail together, it will bring the results into one place, recommend a useful next step and explain how to check whether that step worked.

The workflow in Figure 1 starts with a project, a Python environment and a goal. The user reviews the evidence, makes changes and reruns relevant checks. FixFirst then updates the recommendations and produces a report showing what passed and what still needs attention.

The project will combine knowledge rules, data mining and a knowledge graph. Public PyDFix records, selected BugsInPy defects and controlled examples will support development and evaluation. The intended outcome is an installable local tool, supported by reproducible experiments and a user guide.

Project goals Reduce time spent reading repeated errors, make recommendations understandable, and verify progress using actual check results.

1 Project Background and Market Context

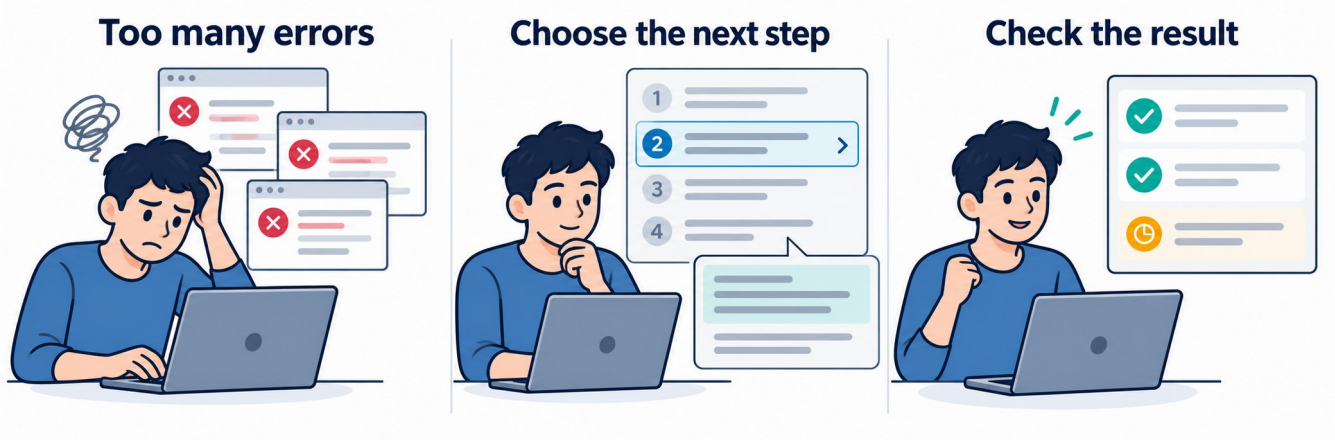


Figure 2 A student moves from a confusing error list to a clear next step

Wang takes over a teammate's Python project before an assignment deadline. He finds a package version mismatch, failing tests and several code warnings. After trying a few changes, he still cannot tell which errors are related or where to start. In Figure 2, FixFirst helps him organise the results, choose an action and check its effect.

Why this problem matters

Existing checks can identify failures, but the user still has to combine their results and decide what to do. Repeated messages may look like separate faults. A successful check may also be mistaken for proof that the whole project is working. These decisions are especially difficult for less experienced developers.

In the Stack Overflow 2025 survey, the question about frustrations with AI tools received 31,476 responses. Of those respondents, 66% cited answers that were nearly correct but still wrong, and 45% cited the extra time needed to debug AI-generated code. These findings motivate a focus on checking advice and verifying results.[1]

The PyConf study also found that package version checks alone can miss problems with the actual running environment.[2] FixFirst will therefore consider project requirements, the installed environment and test results together.

Who would use it

The initial users will be students, junior Python developers and maintainers in small teams. Likely entry points are taking over a project, moving it to a new environment and checking it before submission. Example projects and a free pilot will make the tool easy to try.

The market opportunity is a focused troubleshooting aid that fits around familiar tools. User tasks will test whether it saves effort and whether participants would use it again; the public survey alone does not establish demand for FixFirst.

2 User Needs and Product Features

Figure 3 shows how an action, its supporting evidence and a check button will appear together. Table 1 connects the main user problems to the proposed features.

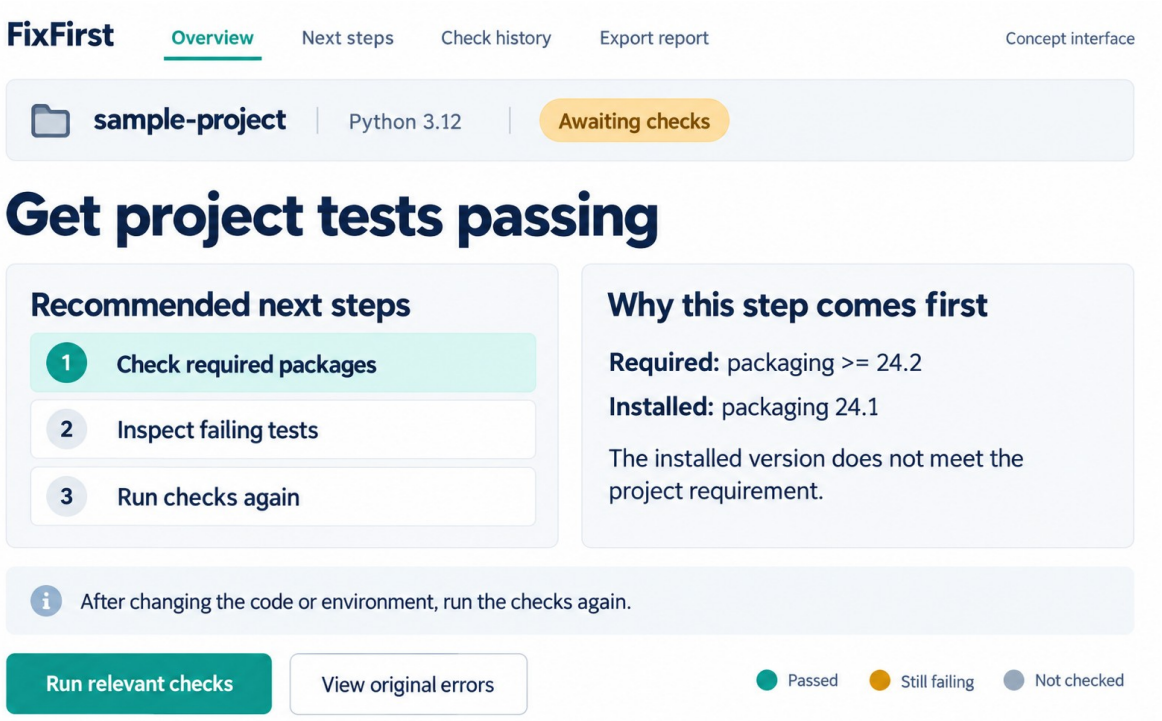


Figure 3 Concept interface with recommended actions and supporting evidence

Table 1 User pain points and the features that address them

Pain point	Feature	User benefit
Results are scattered.	Collect environment, dependency, test and code-check results.	Less switching between tools.
The same error appears repeatedly.	Group similar messages and retain the originals.	Fewer messages to read.
The next step is unclear.	Order actions by the selected goal and required conditions.	A clearer starting point.
Advice is hard to trust.	Show requirements, environment facts and source errors.	Advice that can be checked.
The effect of a change is uncertain.	Rerun checks and update only the covered issues.	Clear passed and unchecked states.
The process is hard to explain.	Export the environment, check history and remaining issues.	Easier help and handover.

A local browser interface and a command-line entry point will share the same workflow. Users will make code or environment changes themselves, then run checks and save the updated report.

3 Market Research and Competitive Analysis

FixFirst will build on established checking tools. Table 2 compares their documented roles with the product's intended focus; the comparison is a positioning analysis, not a performance ranking.

Table 2 Related tools and the proposed focus of FixFirst

Existing solution	Main capability	FixFirst focus
pip[3] uv[4]	Check installed dependencies and manage package versions and project environments.	Combine requirements and runtime results to suggest follow-up checks.
pytest[5]	Run tests, explain failures and rerun selected tests.	Connect test failures with environment evidence and show what a rerun covered.
Ruff[6]	Check code, format files and fix supported issues.	Include code warnings in a wider troubleshooting flow, ordered by the user's goal.
GitHub Copilot[7]	Agent mode can inspect a task, edit files and run commands through multiple steps.	Provide a local core workflow with visible evidence and check coverage, without requiring a cloud model.
Sentry Seer[8]	Use code and runtime information to investigate and fix issues across development stages.	Offer a simple entry point and shareable records for small Python projects.
PyDFix[9]	Detect dependency errors and iteratively patch versions to make Python builds reproducible.	Guide users across mixed checks, with goal-based actions, user-controlled changes and explicit verification.

Choose actions that help the current goal

A user may want tests to load, code checks to pass or the project test suite to pass. The same errors may need a different order for each goal. For example, a missing prerequisite should be investigated before repeatedly running tests that cannot start.

Explain the recommendation and verify its effect

Each recommendation will point to a project requirement, an environment fact or a check result. After a change, the next check will determine progress. Passing one check will confirm only its own scope, while the report will retain unresolved and unchecked items.

The project's contribution is this focused combination of guidance, evidence and verification. Comparisons will test whether it improves the task rather than assuming that the combination is effective.

4 Project Scope and System Design

Figure 4 shows the flow from checks to advice, with new results updating the next step.

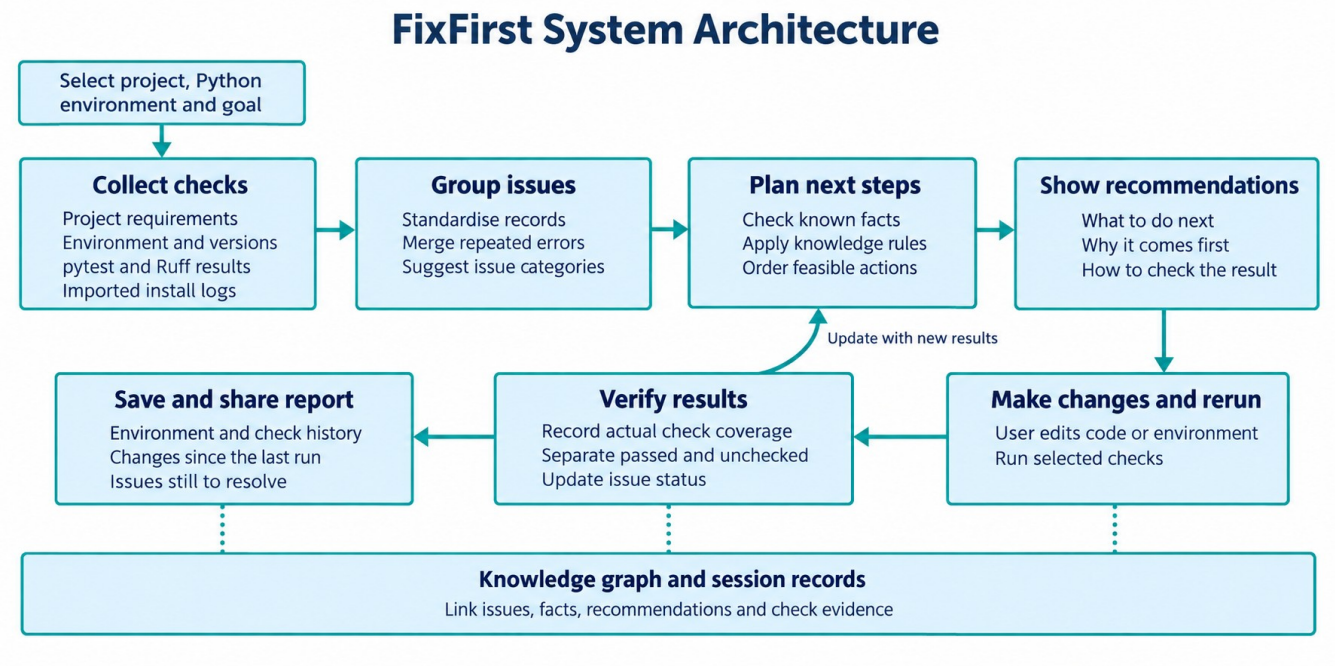


Figure 4 System architecture and the feedback from verification to planning

Turn check results into useful actions

The system will read package requirements, inspect the selected Python environment and collect dependency, pytest and Ruff results. Imported installation logs will provide further evidence. A common format will retain original messages and support grouping.

Rules will combine the evidence and user goal to recommend actions, each with a reason and a way to check the outcome. Users will make the changes and run selected checks.

Use new results to update the plan

Verification will record what ran and what passed, then update issue status and the next step. Users can continue towards the selected goal or save the report to resume later.

Initial scope

The first release will cover small Python projects on macOS and Linux, including third-party dependencies, test loading, test execution and code checks. Shared Python modules will serve the interface, command line and experiments.

Records will stay local by default. Users will see the check scope and can stop execution. Unsupported errors will retain their original messages and be flagged for further investigation.

5 Intelligent Reasoning Techniques

The project will apply three of the course's technique groups.[10] Table 3 links each group to a concrete method and a user-facing function.

Table 3 Course techniques and their role in the product

Course technique group	Methods	Product role
Decision automation and knowledge-based reasoning	Forward chaining; action rules with prerequisites; goal-based heuristic ordering.	Derive feasible next actions from known facts and rank them for the selected goal.
Knowledge discovery and data mining	TF-IDF and cosine similarity; grouping with structural constraints; Gini decision tree.	Reduce repeated messages and suggest likely issue categories.
Cognitive techniques and knowledge representation	Typed knowledge graph; breadth-first traversal; common questions mapped to graph queries.	Link issues, recommendations and evidence to explain what to do and what remains unchecked.

Knowledge rules guide the next step

Forward chaining starts from confirmed facts and applies rules to derive an action. If the installed version does not meet a project requirement, the system can recommend checking that mismatch. Actions must meet their prerequisites; feasible actions will then be ordered by their likely contribution to the goal and estimated effort.

This approach suits a small, explainable set of troubleshooting actions. It also allows the plan to change after every check. The system will recommend a practical next step rather than claim a globally optimal repair sequence.

Data mining reduces repeated information

TF-IDF converts error text into comparable features, and cosine similarity measures how similar two messages are. Checks on the stage and error type will help prevent unrelated failures from being merged. A Gini decision tree will separately use structured error features to suggest a category. Its results will be compared with a rule-based classifier.

A knowledge graph explains the evidence

The graph will connect issues, known facts, recommendations and check records. Breadth-first traversal follows these links to answer supported questions such as why an action comes first or what has not been checked. Confirmed facts and candidate explanations will be distinguished; similar wording alone will not be treated as proof of causation.

The evaluation will measure each module and the complete workflow, so the role of each technique can be assessed.

6 Data Sources and Collection

The main data source will be existing public PyDFix logs and experimental records. BugsInPy will support reproducible checks on real defects, while a limited set of controlled faults will fill coverage gaps. Table 4 explains what each source contributes.

Table 4 Available data and its intended use

Data source	Available material	Use in FixFirst
PyDFix public research artefact[9]	Build logs, dependency-error analysis, candidate packages and repair outcomes. Two published analysis tables contain 506 and 1,421 data rows.	Primary material for parsing logs, studying repeated errors and preparing reviewed dependency-error examples.
BugsInPy benchmark[11]	The original benchmark reports 493 real bugs from 17 Python projects, with tests and tools for reproduction.	Select compatible defects and fixed versions to test recommendations and verification on real projects.
Controlled supplementary cases	Small Python projects with deliberately introduced single or combined faults and a known working version.	Cover gaps such as code warnings, missing environment settings and partial reruns, with known expected outcomes.

What the public data makes possible

PyDFix provides a sizeable starting pool: the two analysis tables contain 1,927 records in total. These are candidate experimental records, not 1,927 independent, manually labelled Python bugs. Related runs and versions may overlap, and the selected material must be checked for language, relevance and duplication.[9]

Saved logs can support parsing and message analysis without rebuilding every historical environment. Live verification will use a smaller subset that can be reproduced reliably. This keeps the data work practical while preserving a separate test of whether a repair really changes the result.

Planned coverage and acceptance conditions

The curated collection will target at least 60 reviewed cases across five areas: environment setup, dependencies, test loading, test execution and code checks. Public cases will be used first, with controlled cases added where necessary. This is an initial evaluation collection, not a claim of broad production coverage.

For live verification, the target is at least eight reproducible real defects across four third-party libraries. A case will be admitted only when the same relevant test fails on the defective version and passes on the fixed version. Historical environments that cannot be reproduced will be reported separately.

7 Data Preparation and Model Training

Public records provide examples of failures, but they do not directly provide every label needed by FixFirst. The project will therefore prepare a reviewed dataset before training the classifier and evaluating recommendations. The data sources in Table 4 will be processed through four steps.

1 Collect and standardise records

For each selected case, the dataset will record its source, upstream project or defect, environment, triggering command and original output. Logs will be converted into a common format while retaining the raw evidence. Download scripts and source versions will make the collection traceable. Reuse conditions will be recorded with the materials.

2 Add and review the labels

The team will label issue categories, messages that belong together, reasonable next actions and the checks needed to confirm progress. Another member will review the labels. PyDFix's suggested packages and reported repair results will be treated as candidate annotations; they will not automatically become the correct answers for our evaluation.

Where several actions are reasonable, the labels will accept more than one valid next step. A historical fix can help establish the cause, but it will not be supplied as an input to the system being evaluated.

3 Train the classifier and prepare the rules

The Gini decision tree will learn issue categories from labelled, structured features such as the check stage, error type and package evidence. TF-IDF will be fitted on training text, and grouping thresholds will be adjusted on separate validation cases. Knowledge rules will be written from documented conditions and reviewed examples.

The project will not train a large language model. The learning component has a narrow task: helping classify errors. Rules and the knowledge graph will organise actions and explanations. When a category is uncertain, the system will retain the original evidence and request a further check.

4 Evaluate on cases kept out of training

Training, validation and test cases will be separated by upstream project or defect group. Versions and message variants of the same fault will stay together. Because PyDFix includes BugsInPy-derived material, overlapping cases will be grouped across sources before splitting the data. Test cases will not be used to tune rules or thresholds.

The final dataset description will report independent defects, repeated runs and controlled variants separately, together with rejected and unreproducible cases. Real and controlled results will also be reported separately so that convenient examples do not hide gaps in real-world coverage.

8 Evaluation Plan and Expected Deliverables

Evaluation will test both technical correctness and usefulness to users. Table 5 sets out the comparisons. All numerical values below are proposed targets; the final report will show measured outcomes, sample sizes and failures.

Table 5 Evaluation questions and planned comparisons

Evaluation area	Comparison	Main measures
Message grouping	Compare with grouping only identical text.	Precision, recall and F1; target grouping precision of at least 0.90.
Issue classification	Compare the Gini decision tree with rules on the same held-out cases.	Per-category results, Macro-F1 and common misclassifications.
Next-step guidance	Compare with message order and a fixed order of issue categories.	Whether the first action is reasonable; invalid attempts before the goal is reached.
Verification	Check full runs, partial reruns and unchecked items.	Correct status updates; no unsupported completion claim in the specified boundary tests.
User troubleshooting	Compare ordinary tools with FixFirst on matched tasks.	Completion rate, time, unsuccessful attempts and understanding of the advice.

User study and outcome targets

Six to eight participants with basic Python experience will complete two tasks of similar difficulty but different content. Tool order will be counterbalanced. The targets are an 80% task completion rate and a 20% reduction in median time for successfully completed tasks, compared with ordinary tools. Failures and timeouts will be reported alongside timing results.

Ablation experiments will remove message grouping, goal-based ordering or evidence display one at a time. These comparisons will help identify which functions contribute to the result. The small user study will indicate usability trends rather than establish an effect for all developers.

A usable product and reproducible evidence

The final delivery will include the installable local tool, example projects, a user guide, data with source notes, reproduction and evaluation scripts, and a project report with demonstration materials. Users should be able to select a project, review advice, make a change, rerun checks and export a useful report.

This delivery will demonstrate business value through user tasks, system design through the reasoning workflow, and implementation quality through an end-to-end product. The experiments will show how the three course technique groups contribute to the outcome.[10]

References

Online sources were accessed on 11 September 2026. Source titles link to the original materials. Superscript citations and figure and table references link within this document.

[1] Stack Overflow (2025). Developer Survey 2025 AI.

[2] Peng, Y., et al. (2024). Less is More? An Empirical Study on Configuration Issues in Python PyPI Ecosystem. ICSE 2024.

[3] Python Packaging Authority. pip check documentation.

[4] Astral. uv documentation Locking and syncing.

[5] pytest developers. pytest documentation.

[6] Astral. Ruff documentation.

[7] GitHub. Asking GitHub Copilot questions in your IDE. Includes agent mode.

[8] Sentry (27 January 2026). Seer now debugs in every stage of development.

[9] UC Davis PLSE. PyDFix research artefact and documentation. Source revision b276d79e7aff. Counts exclude CSV headers and refer to records, not unique defects. BugsInPy analysis table. BugSwarm analysis table.

[10] NUS-ISS (2026). IRS practice module and exam briefing v2.17; IRS practice module project proposal and final presentation guidelines v016. Course materials supplied for the module.

[11] Widyasari, R., et al. (2020). BugsInPy: A Database of Existing Bugs in Python Programs to Enable Controlled Testing and Debugging Studies. ESEC/FSE 2020. Official benchmark repository.

AI assistance statement

OpenAI Codex was used to assist with source research, translation, drafting and document layout. The built-in image generation tool produced the English concept illustrations. The team is responsible for the final content, references and quality. AI assistance used in development and experiments will be recorded separately.